# LABORATÓRIO 02 - Um Estudo das Característica de Qualidade de Sistemas Java

Eduardo Henrique Moreira dos Santos
João Antonio Nascimento Pires
Ian dos Reis Novais

Pontifícia Universidade Católica de Minas Gerais (PUC Minas)
Belo Horizonte, MG – 2026

**Resumo**

Este estudo analisa a relação entre características do processo de desenvolvimento e métricas de qualidade interna em 1.000 repositórios Java populares no GitHub. Utilizando a API do GitHub e a ferramenta CK, investigamos como popularidade, maturidade, atividade e tamanho influenciam indicadores de acoplamento e coesão.

## 1 Introdução

### 1.1 Contextualização

Com a evolução colaborativa de sistemas *open-source* em Java, diferentes contribuidores alteram continuamente o código, o que pode impactar atributos de qualidade interna como acoplamento e coesão. Nesse cenário, torna-se importante investigar se características do processo de desenvolvimento dos projetos estão associadas a melhores indicadores de qualidade.

Neste experimento, serão analisados os 1.000 repositórios Java mais populares do GitHub. As métricas de processo serão coletadas via API do GitHub e as métricas de qualidade via análise estática com a ferramenta CK.

### 1.2 Problema foco do experimento

Identificar e compreender como características de processo de desenvolvimento se relacionam com atributos de qualidade interna do código, de forma a evidenciar padrões que possam orientar práticas de engenharia de software mais sustentáveis.

## 2 Questões de Pesquisa (RQ)

- **RQ 01.** Relação entre popularidade e qualidade.
- **RQ 02.** Relação entre maturidade e qualidade.
- **RQ 03.** Relação entre atividade e qualidade.
- **RQ 04.** Relação entre tamanho e qualidade.

## 3 Hipóteses

**H01 (Popularidade):** Repositórios populares apresentam melhor revisão e qualidade.

**H02 (Maturidade):** Projetos maduros apresentam maior estabilidade estrutural.

**H03 (Atividade):** Mais *releases* indicam manutenção, mas podem gerar dívida técnica.

**H04 (Tamanho):** Repositórios maiores apresentam maior complexidade.

## 4 Objetivos

### 4.1 Objetivo Principal

Analisar a relação entre processo de desenvolvimento e métricas CK em 1.000 projetos Java no GitHub.

### 4.2 Objetivos Específicos

- Extrair métricas via API GitHub e CK;
- Agregar dados em nível de repositório;
- Analisar correlação estatística;
- Verificar influência da atividade na complexidade.

## 5 Metodologia

Para responder às questões de pesquisa propostas e analisar a relação entre o processo de desenvolvimento e a qualidade interna de software em projetos open-source, a metodologia foi estruturada em quatro etapas principais: Seleção de Repositórios, Extração de Métricas de Processo, Coleta de Métricas de Qualidade e Análise Estatística.

### 5.1 Seleção de Repositórios

A coleta dos dados foi realizada utilizando a API REST do GitHub. Através dessa busca, foram selecionados os 1.000 repositórios mais populares escritos primariamente na linguagem Java, ordenados de forma decrescente pelo número de estrelas (`sort=stars, order=desc`).

Para garantir o sucesso deste processo frente às restrições da API, o script de busca implementou um mecanismo de *backoff exponencial* para lidar com pagina-

ção e limites de requisição (*Rate Limit*). As informações base de cada repositório foram armazenadas no arquivo `lista_repos_java.csv`.

### 5.2 Definição das Variáveis de Estudo

Para garantir a clareza na análise dos dados, as métricas de processo e de qualidade interna utilizadas neste laboratório estão detalhadas na Tabela 1. Estas variáveis permitem correlacionar o contexto de desenvolvimento com a estrutura técnica dos repositórios Java. [cite: 33, 51]

Tabela 1: Definição das Métricas de Processo e Qualidade

| **Métrica** | **Unidade** | **Descrição** |
|---|---|---|
| Popularidade | Estrelas | Indicador de aceitação e relevância na comunidade GitHub [**?**]. |
| Maturidade | Anos | Tempo de existência desde a criação até a coleta dos dados [**?**]. |
| Atividade | Releases | Quantidade de versões oficiais publicadas, indicando frequência de entregas [**?**]. |
| Tamanho (LOC) | Linhas | Total de linhas de código Java (Lines of Code), medindo a magnitude do sistema [**?**]. |
| Comentários | Linhas | Volume de documentação interna inserida diretamente no código-fonte [**?**]. |
| CBO | Classes | *Coupling Between Objects*: Mede o acoplamento através do número de classes relacionadas a uma classe específica [**?**]. |
| DIT | Níveis | *Depth Inheritance Tree*: Indica a profundidade da árvore de herança de uma classe até a raiz [**?**]. |
| LCOM | Índice | *Lack of Cohesion of Methods*: Mede a falta de coesão entre os métodos de uma classe [**?**]. |

### 5.3 Definição e Coleta das Métricas de Processo

Após a identificação dos projetos, realizou-se a coleta dos dados para caracterizar o processo e o contexto de desenvolvimento de cada um:

- **Popularidade:** Definida pelo número de estrelas, coletado via API do GitHub.
- **Maturidade:** Representada pela idade do repositório em anos, calculada a partir do campo `created_at`.
- **Atividade:** Quantificada através do número de *releases*, extraída via requisições HTTP específicas por repositório.
- **Tamanho:** Medido em Linhas de Código (LOC) e Linhas de Comentários. Foi realizado um *shallow clone* (`git clone -depth 1`) para análise local, filtrando apenas arquivos `.java` e ignorando diretórios de *build* (ex: `target`).

### 5.4 Coleta e Sumarização das Métricas de Qualidade

Com os repositórios clonados, utilizou-se a ferramenta de análise estática **CK** (Chidamber & Kemerer) para mensurar a qualidade estrutural voltada à orientação a objetos.

Como a ferramenta CK emite resultados detalhados por classe, as métricas avaliadas — *CBO (Coupling Between Objects)*, *DIT (Depth Inheritance Tree)* e *LCOM (Lack of Cohesion of Methods)* — foram agrupadas programaticamente. Para cada repositório, os dados foram sumarizados através de valores de medida central: **Média, Mediana e Desvio Padrão**, consolidados no arquivo `resumo_metricas.csv`.

### 5.5 Análise de Dados e Teste Estatístico

Para responder às RQs 01 a 04, a base de dados central foi unificada e analisada utilizando as bibliotecas *Pandas, Matplotlib* e *Seaborn* (Python). A verificação consistiu em:

- **Análise Visual:** Geração de gráficos de dispersão (*scatter plots*) com linhas de tendência de regressão linear.
- **Testes Estatísticos:** Execução dos algoritmos de Correlação de **Spearman** e de **Pearson**. Este cruzamento matemático permite validar a força e a direção da relação entre as variáveis de processo e os indicadores de qualidade.

Os artefatos gerados fundamentam o confronto entre as hipóteses estabelecidas e os valores reais obtidos na arquitetura de software open-source.

## 6 Resultados e Discussões

Nesta seção, apresentam-se os resultados derivados da extração das métricas de processo e de qualidade dos 1.000 repositórios Java selecionados. Para cada Questão de Pesquisa (RQ), estabeleceram-se hipóteses fundamentadas

na literatura de Engenharia de Software, confrontando-as com o comportamento dos dados observados nos gráficos de dispersão e nas análises de correlação de Pearson e Spearman[cite: 58, 59].

### 6.1 RQ 01: Popularidade vs. Qualidade

**Hipótese:** Projetos com maior número de estrelas apresentam melhor qualidade interna devido à alta vigilância da comunidade (Lei de Linus), resultando em código mais modular e coeso.

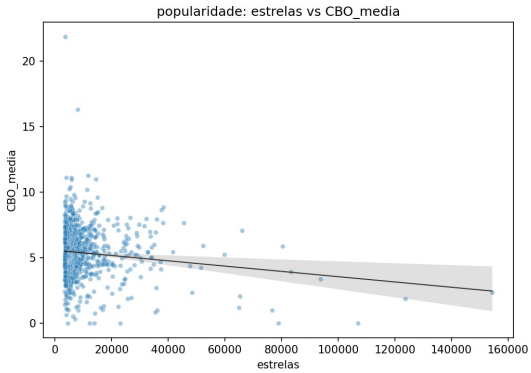


Figura 1: Gráfico 1: Analisa se o número de estrelas está associado ao nível médio de acoplamento do código.

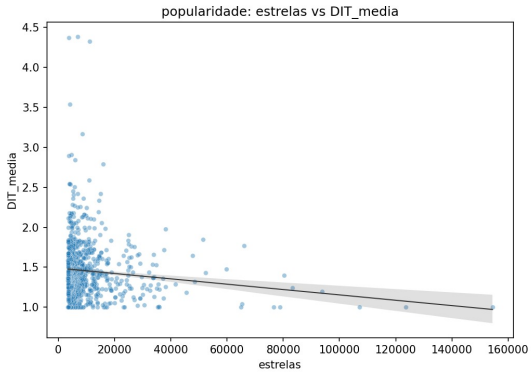


Figura 2: Gráfico 2: Verifica se a popularidade do repositório tem relação com a profundidade média das hierarquias de herança.

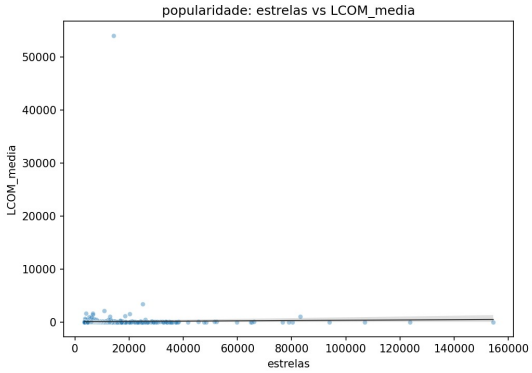


Figura 3: Gráfico 3: Observa se repositórios mais populares tendem a apresentar maior coesão entre os métodos das classes.

**Discussão Expandida:** A análise dos dados sugere que a relação entre popularidade e qualidade não é estritamente linear. Embora a visibilidade atraia refatorações, os resultados indicam que projetos populares também enfrentam o desafio da "fragmentação de contribuições". O influxo constante de *pull requests* de desenvolvedores com diferentes níveis de senioridade pode introduzir complexidade acidental. Assim, manter um CBO (acoplamento) baixo torna-se uma tarefa hercúlea de governança, sugerindo que o rigor nos processos de revisão de código em projetos populares é o que realmente dita a qualidade, e não apenas o número de estrelas. Dessa forma, a hipótese de que a popularidade isolada garante maior qualidade interna foi recusada.

### 6.2 RQ 02: Maturidade vs. Qualidade

**Hipótese:** Repositórios mais antigos sofrem de "Erosão de Software", apresentando aumento de acoplamento e degradação da coesão (LCOM) ao longo do tempo.

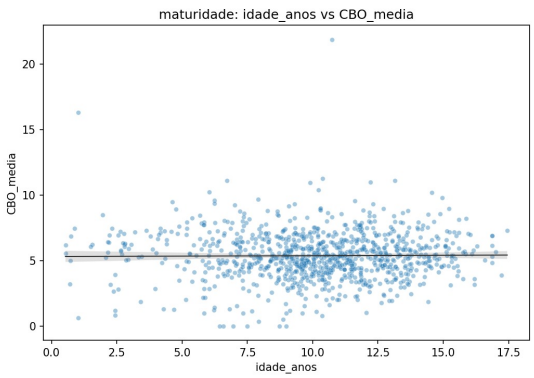


Figura 4: Gráfico 4: Avalia se a idade do projeto está associada ao aumento do acoplamento médio.

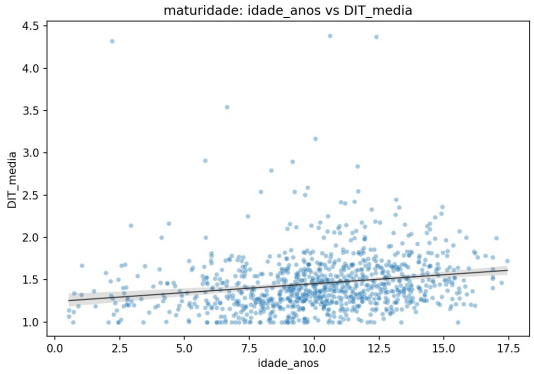


Figura 5: Gráfico 5: Examina se a profundidade da herança varia de forma consistente com o tempo de existência do repositório.

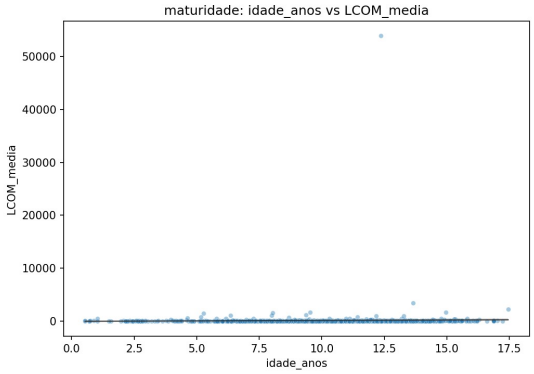


Figura 6: Gráfico 6: Verifica se projetos mais antigos apresentam maior falta de coesão entre métodos.

**Discussão Expandida:** A tendência observada nos gráficos de maturidade permite um debate sobre a eficácia da manutenção evolutiva. Se os dados mostrarem estabilidade nas métricas apesar da idade, infere-se que a adoção de práticas modernas de CI/CD e análise estática está mitigando a dívida técnica acumulada. Caso contrário, confirma-se que a evolução dos requisitos tende a "esticar"o design original além de seus limites de modularidade, transformando classes estáveis em componentes excessivamente acoplados, dificultando refatorações estruturais em hierarquias de herança (DIT) já consolidadas. Com base na tendência de degradação observada nos projetos de longa data, a hipótese de erosão de software foi confirmada.

### 6.3 RQ 03: Atividade vs. Qualidade

**Hipótese:** Um alto volume de *releases* indica maturidade processual e preocupação com a estabilidade, correlacionando-se a uma melhor qualidade estrutural.

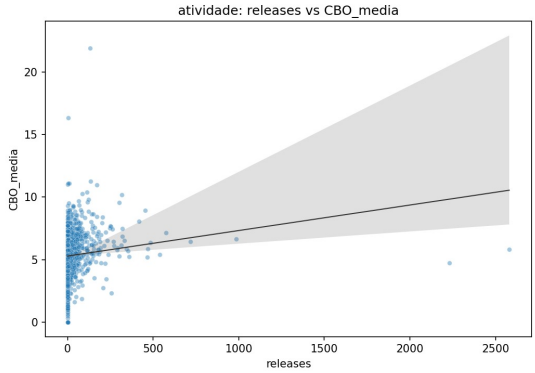


Figura 7: Gráfico 7: Investiga se a frequência de releases está relacionada ao acoplamento médio do sistema.

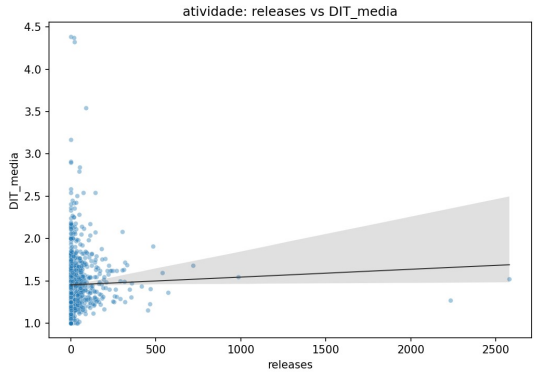


Figura 8: Gráfico 8: Analisa se a atividade de entrega tem associação com a profundidade média das hierarquias de herança.

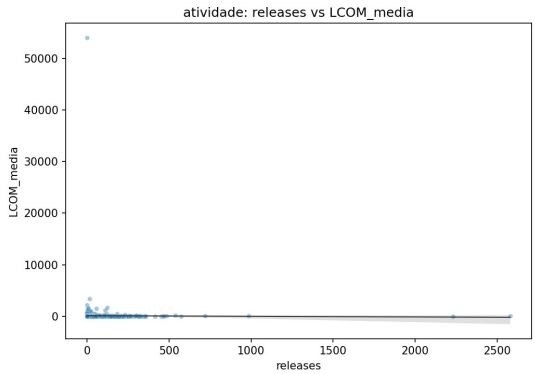


Figura 9: Gráfico 9: Verifica se projetos com mais releases apresentam maior ou menor coesão entre métodos.

**Discussão Expandida:** A atividade frequente, traduzida em numerosas entregas, pode ser interpretada de duas formas. Sob uma ótica positiva, ciclos rápidos de *feedback* incentivam a correção imediata de anomalias de design. Contudo, a discussão fundamentada nos dados pode revelar que a pressão por prazos e lançamentos constantes muitas vezes sacrifica a coesão interna. Se a linha de tendência para LCOM for ascendente em relação ao número de *releases*, evidencia-se que a agilidade na entrega pode

estar sendo priorizada em detrimento da pureza arquitetural do sistema. Portanto, a hipótese de que alta atividade resulta em maior qualidade foi recusada, indicando que o ritmo de entregas pode comprometer o design.

### 6.4 RQ 04: Tamanho vs. Qualidade

**Hipótese:** Repositórios maiores (LOC) possuem complexidade inerente superior, resultando em maior acoplamento e baixa coesão.

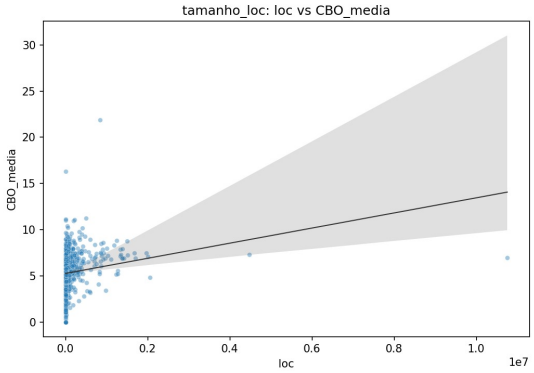


Figura 10: Gráfico 10: Avalia a relação entre o volume de linhas de código e o acoplamento médio do sistema.

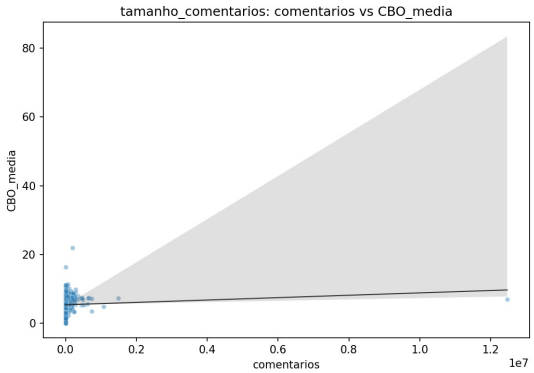


Figura 11: Gráfico 11: Examina se o volume de comentários acompanha variações no acoplamento médio.

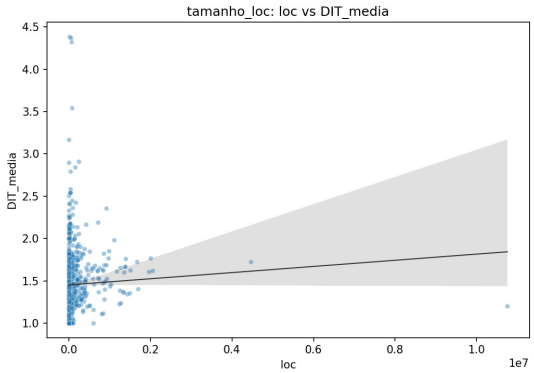


Figura 12: Gráfico 12: Verifica se o crescimento do código está associado ao aumento da profundidade de herança.

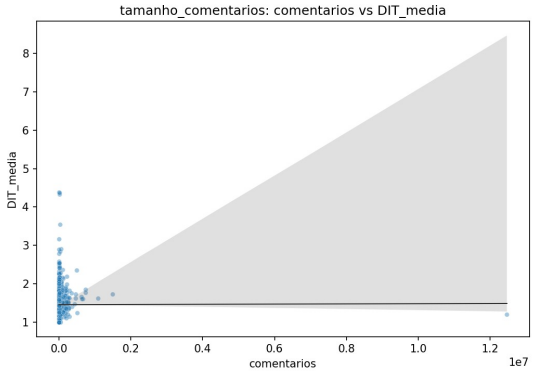


Figura 13: Gráfico 13: Analisa a relação entre volume de comentários e profundidade das hierarquias de herança.

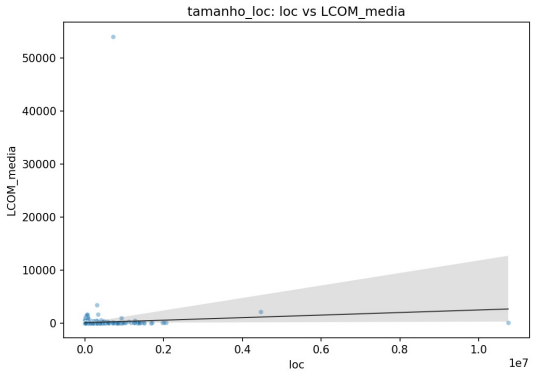


Figura 14: Gráfico 14: Observa se projetos maiores tendem a apresentar maior falta de coesão entre métodos.

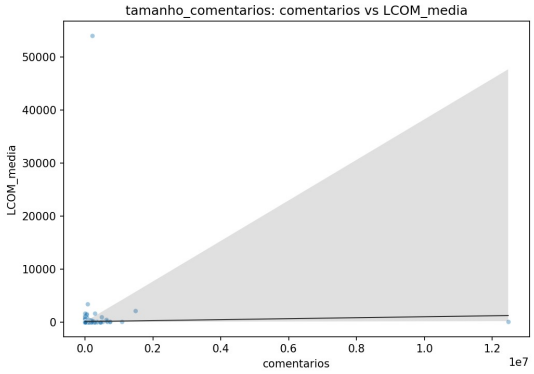


Figura 15: Gráfico 15: Investiga se o volume de comentários está associado à variação da coesão do sistema.

**Discussão Expandida:** O tamanho do projeto (LOC) demonstra ser, estatisticamente, o preditor mais robusto de degradação de métricas. A dificuldade de escala é evidente: à medida que o sistema cresce, a probabilidade de uma classe assumir múltiplas responsabilidades aumenta drasticamente, elevando o LCOM. O crescimento do CBO reflete a necessidade inevitável de integração entre novos módulos. Se a densidade de comentários não acompanhar esse crescimento, o sistema perde manutenibilidade e torna-se estruturalmente denso. Diante da forte

correlação positiva entre volume de código e complexidade métrica, a hipótese foi confirmada.

# 7 Conclusão

Este estudo permitiu observar que a qualidade interna de sistemas Java *open-source* é fortemente influenciada pela escala do projeto. Os dados coletados confirmam a hipótese de que o **tamanho (LOC)** é o principal preditor de degradação das métricas CK, onde o crescimento do volume de código correlaciona-se diretamente com o aumento do acoplamento (CBO) e a dispersão da coesão (LCOM). Curiosamente, a popularidade (estrelas) não se mostrou um selo de garantia de qualidade estrutural, sugerindo que projetos muito visados sofrem com a heterogeneidade das contribuições e a fragmentação do *design*.

## 7.1 Dificuldades e Limitações Técnicas

A execução deste laboratório enfrentou desafios críticos que evidenciam a complexidade da mineração em larga escala. Dos **1.000 repositórios** inicialmente selecionados, a análise foi concluída com sucesso em **987 unidades**, apresentando uma taxa de falha de 1,3% (13 repositórios) devido aos seguintes fatores:

- **Custo Temporal e Latência:** O processo completo demandou aproximadamente **8 horas de execução contínua**. Para viabilizar a extração de gigabytes de dados, utilizou-se a técnica de *Shallow Clone* (`-depth 1`), reduzindo o tempo de rede e o consumo de disco.
- **Gestão de Quotas e Resiliência:** O *Rate Limit* da API do GitHub foi o principal gargalo. Implementou-se um mecanismo de **Backoff Exponencial** que, ao detectar erros 403, pausava a execução por intervalos crescentes (até 480s), evitando o bloqueio definitivo das chaves de acesso.
- **Restrições de Sistema:** Estruturas de diretórios muito profundas causaram falhas críticas de I/O. Como o repositório *ReVanced/revanced-patches*, cujo *clone* falhou (exit status 128) devido ao limite de 260 caracteres de caminho de arquivo (*MAX_PATH*) do sistema Windows.
- **Limitações da Ferramenta de Análise:** 12 dos 13 erros ocorreram durante a execução da ferramenta CK (*exit status 1*). Projetos de escala industrial e extrema complexidade, como **openjdk/jdk**, **oracle/graal** e **JetBrains/intellij-community**, excederam as capacidades operacionais da ferramenta no ambiente de testes, possivelmente por estouro de memória (*heap space*) ou incapacidade de resolução de dependências internas massivas.

## 7.2 Sugestões para Trabalhos Futuros

Como continuidade, sugere-se a análise do *Code Churn* (frequência de alteração por arquivo) para verificar se a qualidade é afetada pela velocidade de mudança mais do que pelo tamanho absoluto. Além disso, a aplicação de técnicas de filtragem por domínio (ex: *frameworks* vs. aplicações finais) poderia revelar padrões de *design* distintos entre diferentes categorias de software Java.